Advanced micro lab

Lab report 4

Asynchronous FIFO with Formal Verification using SymbiYosys

Shams Tarek

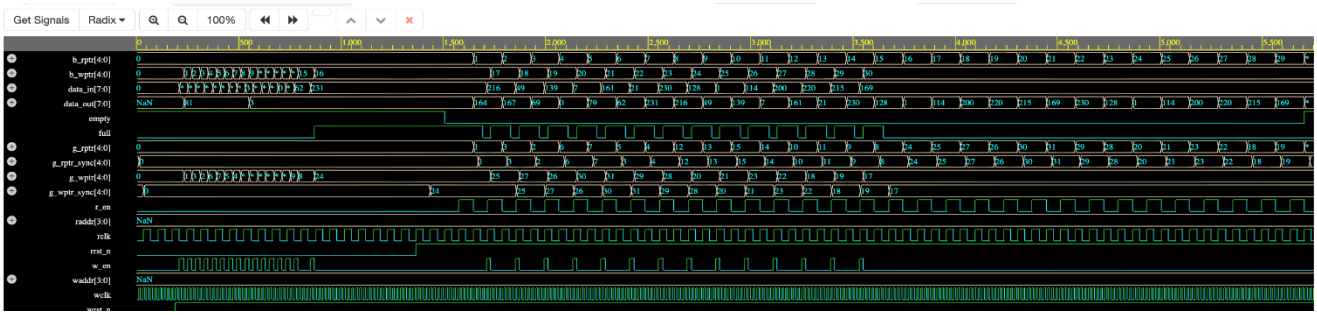
Aya El Bakri

Hanna Mounir

Task 1 – Implementing the Asynchronous FIFO

A self-checking Icarus Verilog testbench that exercises the following scenarios:

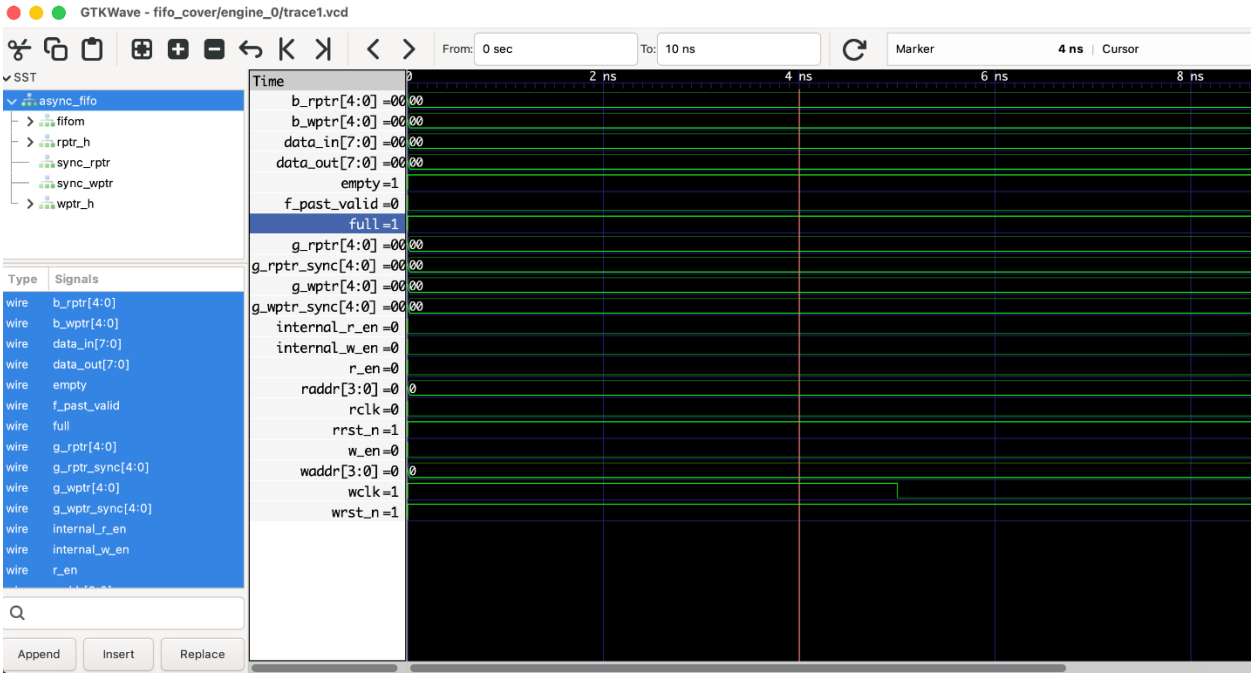
- Sequential write until FIFO full, then read until empty.
- Simultaneous read and write on independent clocks.
- Write attempts when full (wfull asserted) – verify data is not corrupted.
- Read attempts when empty (rempty asserted) – verify no underflow.



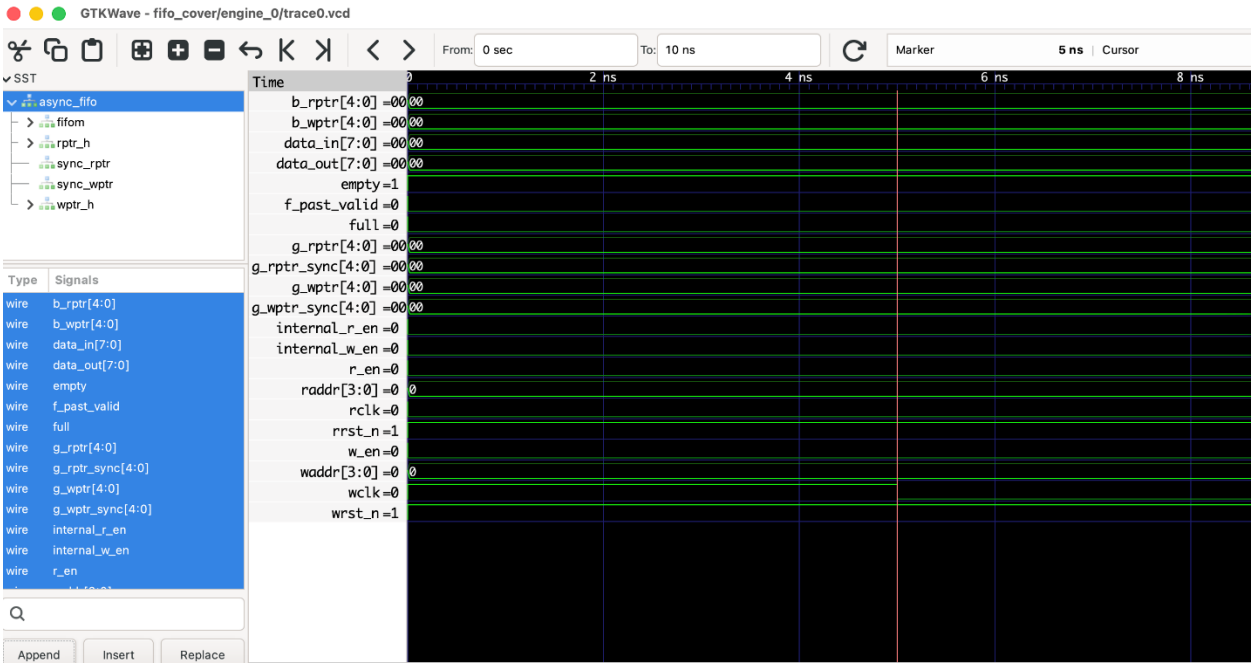
Task 2 – Formal Verification with SymbiYosys

1. Add the following SVA assertions inside async_fifo.v (inside // synthesis translate_off / // synthesis translate_on guards):
   - assert: FIFO never writes when wfull is asserted.
   - assert: FIFO never reads when rempty is asserted.
   - assert: write pointer never exceeds FIFO depth (2^ADDR_WIDTH).
   - assume: wen is never asserted when wfull is high (constrain environment).
   - assume: ren is never asserted when rempty is high (constrain environment).
   - cover: FIFO can become full (wfull reaches 1 from empty).
   - cover: FIFO can become empty after being full (rempty reaches 1 after full is 1).

FULL COVER GTKWAVE TRACE



EMPTY COVER GTKWAVE TRACE



Conclusion

In this lab we implemented an asynchronous FIFO and performed formal verification using SymbiYosys. The design was validated through a self-checking testbench that exercised key scenarios including sequential write/read operations, simultaneous accesses across clock domains, and data integrity checks during full/empty conditions. Formal verification incorporated SVA assertions, assumptions, and cover properties to ensure correct FIFO behavior, with GTKWave traces confirming successful coverage of full and empty states. The asynchronous FIFO meets all requirements, with both simulation and formal verification confirming correct operation across clock domains and edge cases.

Task 3 – GitHub Repository and Verification Report

- https://github.com/ArchitecturebBrown/shams_sem_10_projects.git